

HEXACON2023 - Exploiting Hardened .NET Deserialization by Piotr Bazydło

HEXACON2023 - Exploiting Hardened .NET Deserialization by Piotr Bazydło - Hexacon, YouTube.

- Published: 2023-11-06
- Original: https://www.youtube.com/watch?v=_CJmUh0_uOM
- Preserved from: https://www.youtube.com/watch?v=_CJmUh0_uOM (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

HEXACON2023 - Exploiting Hardened .NET Deserialization by Piotr Bazydło

- Video: https://www.youtube.com/watch?v=_CJmUh0_uOM
- Channel: Hexacon
- Published: 2023-11-06
- The media file itself is not archived; this is its metadata and transcript.

Description

https://www.hexacon.fr/conference/speakers/#dot_net_deserialization

Transcript

Retrieved with yt-dlp in a container, which is the only route YouTube still answers. Expect transcription errors in type and method names.

hello everybody my name is p basid I'm a vulnerability researcher at Trend Micro zero day initiative and today I'd like you to present my research concerning net dis realization and serialization so basically I'm going to show you how I was approaching some harder net targets so first of all I have to mention that I have material for like a two hour long talk and of course you don't want me to talk for two hours so I have written the white paper it will

be public in two days so during the talk I'm going to present you some general ideas and some examples but if you are interested into the heavy heavy technical details do you want if you want to see the entire code flow for vulnerabilities or gadgets you can just reference the white paper later uh so when I'm thinking about hardened dis realization targets I thinking about targets where you for instance can targeted control the target distalization type but you have for instance an extensive block list or um some Alo list sometimes they are very like restricted you can only der serialize a couple of types sometimes they are more like flexible allow you to uh deserialize everything from a given names space for instance so it really depends on the Target right so during the talk I'm going to show you some new ideas that I think that haven't been presented before especially for net and also I will provide you some ideas for your research how you can look for custom gadgets chain of gadgets and stuff which I hope that you're going to find interesting so the first use case that I would like to show you is the solar winds platform rabbit and calization so I have um written a blog post which was published like three weeks ago and I have described a couple of vulnerabilities in solar winds so two of them were based on this rabbit andq vectors so basically the attacker was able to send json. net serialized payload to the solar winds rabbit mq instance and there was a solar winds information service which was taking that payload and was deserializing it so basically there were two separate vulnerabilities because we were able to T control the target serialization type in two different ways also the type name handling for json.net was set to AO it basically me that you were able to fully control the target type so we said that we want to deserialize object of this type right so it was an easy remote code execution here so for it's an fragment of the payload right in the TK you provide the system object for instance in the v provide the gadget and that's it so during the first part of the talk I would like to focus on the patch and how I have bypassed this patch in a couple of ways so they have for this particular attack Vector they have implemented very extensive block list it contains around 30 classes that's a lot it implements a lot of gadgets for I think all the gadgets from y serial.net so the state-of-the-art tool and also gadgets that were mentioned the Friday the 13 Jon attacks black hat USA white paper written by Alvaro Munos and Alexander miros I'm sorry if I have misspelled the names so when we have a quick look at it we see no bypasses right there are no gadgets that we can use against that you know end point and it made me wonder you know because for instance if you are talking about Java dis realization we were mainly targeting you know classes we were looking for Gadgets in third party libraries because there were almost no Gadgets in the Java code itself right in that NET Framework it was completely opposite we had a lot of Gadgets in the doet framework itself so I think we are not looking for Gadgets in different places right because we didn't have to but right now V vendors developers are more and more aware about those bags so we have those block list right which block basically everything that we currently know so my idea is maybe we can look you know for Gadgets in third party libraries maybe we can look for the gadgets in the product codebase itself solar winds is huge right there's a lot of there's thousands of classes so many ideas appear in my head and basically I refuse to think you know that this patch is enough right so the first thing we want to do before looking for some you know custom gadgets for a given serializer we have to be fully aware of the serializer capabilities so here we are targeting json.net and in

general you know you want to avoid a situation in where where you are aware of like 90% of capabilities of the given serializer you are looking for some custom gadgets and it turns out that you would find one but it's in those 10% of capabilities that you don't know right so the more you know about the serializer the better so for instance json.net of course as every Setter based serializer can you know initialize the object for no argument Constructor which is public and call public Setters but it also supports realizable interface and it's Constructor the one which um accepts realization infant streaming context for arguments is supports some the realization callbacks and also it has one very nice property which I find that we have been missing as Community for a long time it allows you to call public Constructor with arguments if it is the only public Constructor implemented for the class so I've been abusing this capability a lot during the entire research so before I even go for you know custom gadgets and stuff I've been reviewing this block place and I have found one weird item it's Microsoft partial editor I was not aware of such gadgets so I started digging and I realized that there was a mistake in the list right the dll name used was used instead of the class so the class should be text formatting run properties and such mistakes happen for instance I think there was a typo in the exchange block list some time ago so yeah but okay so we right right now we know that we can in fact use text for martin part is Gadget which is known but if we go to the yo serial.net documentation you can see the list and in formatters you see no json.net right and at this point you always want to verify that because when somebody contributes to y.net the researcher does not have to you know look at all serializers there's a lot of them so as we are aware of capabilities of json.net right we can always do that by ourselves and I've checked this text formatting run properties class and it quickly real I quickly realized that you can in fact uh deserialize it with json.net because there is this serializable Constructor which you can reach and it just leads to exam pars method which exam leer pars which leads to easy remote code execution in NET Framework so actually it was a first bypass and it was based on you know the mistake in the list but the gadget was not implemented so you always want to ver ify that right if you're aware how the serializer works it should be easy but I like to think about the potential patch when I find the vulnerability and in here it was quite obvious for me that vendor probably is going to just fix this block list item right because okay there was a Mystic in the list but the list is still good I mean as long as you can say that block list is good because in general we disapprove the usage of blocks right it's not the happiest approach for fixing of such bags so at this point I started to look for custom Gadgets in solar wind codebase and uh I was taking you know the capabilities of json.net I was looking for classes that can be deserialize deserialized with those capabilities and I was reviewing them right a lot of like at this point it was a lot of manual work and I have found this worker controller WCF proxy class and as you can see it has only one public con trctor which can be reached through the distalization and it has a very interesting method call it's Louch worker process so it's sounds promising right so at this stage the first thing you want to do is you want to verify whether you can deserialize the arguments so if you can control the arguments to the Constructor and of course you can control the string which is the last argument but we have a service operation mode which turned out to be an Anum so it's easy to deserialize it but we also have the worker configuration so we we have to have a look at this if it turns

out that you cannot control some of the arguments you can always provide the null value so the the serialization will call the Constructor but the argument will be equal to null so if it's relevant for your Gadget the gadget won't be valid sometimes you can just omit some arguments so it's okay so it it turned out that worker configuration can be deserialized because it um defin is a public no argument Constructor and it has a very promising member names you can see the common line member and the comment arguments member so we are investigating further and the lunch worker process initialize the process start info which is again great for us and then it just starts a new process so the only thing we have to do is we have to verify whether we control this process start info or not and again it turned out that we are able to control the process start info so uh at the first place we are controlling the argument the first part of the argument so we can provide anything it uh here and escape the rest of the arguments with erson for instance and then we've got a puth combine where the SEC which creates pu to the execute table so we control the second argument to the puff combine so we can provide an absolute puff here or per do the path traversal it doesn't matter so in fact it turned out that here I have designed like the first way to bypass the the entire block list with the custom Gadget which existed in the solarin codebase you can see the type worker controller WCF proxy and common line here I've just used the puff traversal to reach the CMD exit and I provided arguments so there is one more important thing about custom gadgets if you're for instance a red Timmer and you want to be stealthy so you can use those custom gadgets to you know in the code base to go under the radar right because a lot of solutions like web application firewalls detects you know the gadgets that are commonly known if you use such a gadget nobody knows about it right it's in the product code base so there's a pretty huge chance that you're going under the radar here okay so I'm going to scbe the third bypass because it was mainly based on the previous one so we can find the details in the white paper but now I've reached my favorite Gadget to be honest because we tend to look something to look for something that gives you a direct remote called execution like the previous Gadget right it was simple because it allowed us to start a new process it's the most basic thing that you may look for but uh sometimes you won't be able to find those but you may be able to find something that is based on like on a product functionality for instance so imagine a this realization Gadget that I don't know creates a new user allows you to execute a new SQL query right it allows you to modify product configuration you never know what you will find in the product code base and all of these may appear to be useful so in here I have a very interesting Gadget in solar WIS the class name is credential initializer which is interesting like has interest interesting name and we've got the input argument called loog config file so there was a lot of code down beneath but what we have to know that we are reaching this part of the code and basically this gadget allows you to provide a file the path to the file with the log foret configuration and it allows you to modify on the Fly the configuration of log foret for solar winds so L for net is like you know L forj but for net right uh and right now we have to think how can we deliver the file so it's windows so we have two ways the first one we can provide the UNCC puff and load file directly from your SMB server solar WIS is rather the uh internal product it's not exposed to the internet and the majority of the environments you should be able to use uh internal environments you should be able to use your s SMP server to deliver the file other way is

you know to chain it with legitimate file right functionality like for instance upload of image functionality you know there is no verification of extensions here or whatever so if you're able to upload anywhere to the file system you can just change it but I have just used the and CPU and remotely loaded the file so at this stage we have to think how can I abuse the gadget that allows me you know to modify the log foret configuration and if we are thinking about logging my first idea was like file right capabilities because we are writing something to a file so I've studied the loog for net configuration and I've quickly found on such a configuration so for the file tag in the loog foret configuration we are providing in the path so as you can see uh I was trying to create a webshell the header uh specifies the thing that the first thing that is going to be written to the file when it's been created right so in the header we can just provide our webshell payload and the third part is the conversion pattern so here it is set to null and it's very important because solar Wings when you modify you know the configuration it's trying to write stuff to your file and it's logging stuff all the time but when you specify the null conversion pattern it will always try to you know write empty string to the file so it won't mess with the file that you have uploaded so you can in such a way you know you can just write any file and it was another way to bypass the block list with the custom Gadget and this time you know it was like not an obvious one right so keep that in mind because if you create a tool for instance something based on graph searching or whatever uh for which may be used for finding new gadgets it's very good right but all there is a huge chance that you can skip such a gadget right so it's always good to also prefer the manual review of classes that can be used okay so that's it for solar WIS for now and right now I would also like talk for a second about the exchange and the proxy nutshell because I have found a lot of exchange vulnerabilities that were based on this attack vector and I would like to present you quickly two rce vulnerabilities in exchange because here we are abusing both allowed and blocked classes so with custom exchange let's say code base so I think it's pretty interesting and the entire proxin not shell Vector is pretty complex so I'm not going to describe it in details but in order to understand the St that's happening here you have to be aware of two fundamentals the first thing is that you know the pro that shell uh during the proxy nutshell distalization there are like two two different deserializers the first one is a custom exchange deserializer which allows you to retrieve a type right so we are just retrieving a type and this type was not controlled in any way and then this type was passed to another Powershell related deserializer which was pretty powerful and allowed you to do a couple of things so the first thing was that you could provide any type and you could call a public single argument Constructor for this type and control the input arguments also it allowed you to call a public static Pur method so the exploitation for the proxy not shell if you were aware of all those things was pretty simple you could just provide exam reader class and call it spars method and it was an easy remote code execution right and that's it so again I was thinking about the patch and as expected the type control was implemented so basically uh Microsoft implemented and a lot of Alo lists for the type and we were not able to deliver any type so my idea here was I'm going to review those allowed classes and see whether some of them can be abused and there was a different list just for allowed generic classes and it seem to be a particularly interesting Target because you know generic classes in net allow you to Define some internal class that is going to be used somehow uh

when somehow in this targeted class right so in here I want to focus on multivalued property and dagnet multivalued property classes basically dagnet multivalued property extends multivalued property and it leads to the same root cause and the same code so you can say that basically they are very similar so what happens with this multivalued property so it defines a public single argument Constructor which we can call with um exchange dis realization and there is a lot of code down there but we reach we can reach triar conversion method and TR Constructor conversion method which sounds pretty familiar right because in general those two methods allow you to have this inner type defined and take the type and calls call the pur string method or call the single argument Constructor for the given internal type so it's basically like the inner dis realization implemented in the multivalued property and it's very similar to the one implemented in the Powershell remoting but just a little bit less powerful so in to sum up you can do something like this you are calling a Constructor for multivalued property when the um inner type is set to exam reader and as an input to the Constructor you deliver the exam Gadget right and it in fact leads to the exam leader pars method call where we fully control the input to the method so in such a way I was able to take the allowed class which implements an internal dis realization which had no type you know verification implemented and I was abuse able to abuse it to get the remote code execution so after that again we can think about the patch and my idea was that multivalued property is not going to be removed from the list of allowed classes because it's been heavily used by The Exchange internally but they are probably going to implement a control of this inner type that we are providing to the multivalued property and basically I was right but there was one difference ah okay so I also wanted to show you I just go back to the multi valid property for a second because as you can see in the payload we are defining multivalued property with the reader so the exchange says that multivalued property is allowed so it's okay right but it doesn't verify the internal type right and we have this um patch for the internal type control for the multivar property uh but there is one issue the inter the control for the internal types for the multivalued property is not based on the white list but on The Blacklist right and it again give you a field for misuse so exchange block list is pretty extensive it contains thousands of different classes but again my idea here was so let's look for the classes inside of the exchange right because there were no almost no internal exchange class in the block list so maybe I will able to do to find something and I have quickly found the commment class so as you can see the commment class just implements a single argument Constructor which allows you to start a new process and that's it right so again in order to bypass the patch I was defining the multivalued property the inner type was set to comment and as an input to the Constructor I was able to you know Define any command that I want to run and that's so I was abusing both the you know allow list and the block list at the same time so I was using the allowed class to deliver the inner class which was not blocked right so in the demo uh you can see that I'm running the exploit so um those vulnerabilities can be exploited by any user with the mailbox in the exchange so basically by almost any domain user so we have reached the multivalued property and we have reached the try Constructor conversion and as you can see our Target type is common here right so we have reached the reflection which actually calls our Constructor we have reached the command uh the command Constructor and

you know we have the definition of the process start info and if we continue the C gets popped so we got the remote code execution in exchange okay so right now I would only like to quickly mention the gadgets in third party Library so I have mentioned that also we may look for such gadgets and I don't want to spend so much time on them right now you can find details for all the gadgets in them in the white paper but I only want to mention a couple of things so I didn't look at a lot a big amount of libraries only I have taken some popular libraries from nuget and the I've taken the libraries which were used in the products that I were targeting at that time so for instance you can see that there are free remote dll loading gadgets and what's important is that two of those libraries can be used to get the remote code execution not only against NET Framework but also newer versions of it like Net 5 six or seven uh and also I would like you to if you're are interested into that stuff you can have a look at the pre preserve working folder Gadget you know it's very interesting one because it allows you to modify a current directory for the process and completely mess with the file based operations which are based on relative paths right because you can provide a different current directory and it's pretty fun and there's probably a lot of ways to abuse that in huge applications uh one also thing to consider is that if you are concerned about the payload length sometimes it's relevant sometimes not the real remote loading dll loading gadgets are extremely short you know so if you're concerned about length and they are available you may want to use them okay so now right now I would like to switch to a different uh to a different topic and it concerns Delta electronic infras device Master product it's some IC scab based product which is used by some companies and in general what you have to know is it had a lot of security issues in the past so some vulnerabilities were reporting to reported to the zdi program by researcher known as Kimia and another anonymous researcher and it was listening on two different TCP ports and there was now authentication implemented to this communication Channel and it was based on binary format dis realization so you could just you know deliver any binary format or Gadget and just get a code execution like that so it was really simple so so time passed by it was patched and we have received receive no bypasses for the patches and I was interested how they are going to know to fix that so I've decided to take a look and it turned out you know that the entire architecture has changed so first of all the binary formatter was switched to message pack serializer and I was like what the hell is message pack I was not aware of that then there was some dis realization and it was very fine credential so they have implemented the authentication and I have actually bypassed the authentication it was a very fun bag because it was based on the fact that you were able to control one variable which you should not be able to control and you could completely mess with the authentic implemented authentication logic so this bug is described in the white paper you can have a look but what is important for us is that the entire dis realization has been happening you know before the authentication because even the authentication mechanism was based on this this serialization then it was serializing the object again and sending to you so okay we've got this message serialization so I was doing this research in November 2022 and this serializer was rather unknown to our community it has changed a little bit in March this year because um another researcher has made a comment to is serial.net and has implemented the object data provider for message pack so but when I was analyzing it there was no public

information regarding exploiting the message pack so I had to you know again you have to be familiar with the serializer and know all of its capabilities so it turned out to be ridiculously powerful you know so it is of course as every said based serializer allowed you to call a no AR Constructor with public Setters but it also allowed you to call Constructor with arguments however it's sometimes very hard to control uh it allowed you to set members through a reflection which is extremely powerful and it also allowed you to call Private Setters for instance which is pretty unusual also it implements something called typeless mode which is a little bit like Json type name handling which when you have some generic member like object you can deliver an arbitrary type and it will be our Target Target distalization type right so we can control the type in such a situation and this typeless mode was in fact used in message pack but what is very important about the message pack it tends to change its def Behavior between versions so you have one version of message pack and it works in such way then you switch the version and it behaves completely differently right so you have to consider that when you're dealing with message pack and as I have already mentioned the object data provider so the commonly known Gadget implemented in y serial.net should work for message pack yes but it only works from version 2375 because you know the default Behavior has changed in that version and infras suit device master was based on some ancient version where object data provider was in fact not an rce Gadget but the denial of service Gadget so it's connected with the fact that it's even if you didn't provide a member it was setting the member to null and it was modifying one of the members that you couldn't control and object data provider was just killing an entire process so we can find all of the details in the white paper but at that point I was like okay it doesn't matter there's no gadgets that I can use against this message pack I'm again going to look for some custom dis realization gadgets right and it turned out you know that the code base is not so huge and I failed to do so so I was not able to find any dis realization gadget for the me for the message pack that I could use against this product but another idea came to my head and it was you know insecure serialization because as I have already mentioned uh you are sending the serialized payload to the message to the infras device Master it deserializes it and then it serializes it again and I find this scheme maybe not common but present in some products especially if the entire Communication channel is based on the dis uh on the dis realization so my idea was this we are sending some Mal malicious object to the Target the this realization is being performed but it doesn't lead to anything malicious right because you know it's not a valid der serialization Gadget so the product May perform some operations on the object or not but it serializes the object again and during the serialization we are calling Getters right not Setters but Getters and my idea was to look for you know the serialization the serialization gadgets where something interesting and potentially malicious happen in the gter call and in order you know it was a conception right uh so in order to prove that it may be applicable we have to look for gadgets so of course we can look for Gadgets in three different ways we can look for sur realization Gadgets in NET Framework we can look in third party libraries and we can look in the product code base and in fact the product code base is a very promising attack Vector for looking for info League gadgets and I will mention that uh later and also I have distinguished two different kind of gadgets here the first one is the serialization gadget which where malicious stuff

happens during the get call right but also you can have a deserialization to serialization gadget and in that example the malicious stuff may happen during the deserialization but you are not able to benefit from that if there is no serialization so I will explain it on an example soon and I've been able to find six gadgets free in .NET Framework and two of them directly lead to the binary formatter deserialization so you have a getter which leads to Binary format deserialization also I have found like free Gadgets in third party libraries so what you have to know about the insecure serialization that sometimes depending on the deserializer it may be extremely hard to exploit that so for instance here we've got the settings property value Gadgets in .NET Framework you can see that we've got a property value get and it leads to Binary format deserialization and we are able to control the stuff that is passed to the binary format deserialization so it's okay and it works really good for message pack at least for some versions but for json.net is problematic because uh during the deserialization phase you are not able to control all the members and before we reach this get property value during the serialization json.net it's going to call a different getter for the member that we are not able to control with json.net and the object reference exception is going to be thrown so if there is no exception handling defined for json.net it may be hard to abuse that and another example in .NET framework is the security exception where again the get method leads to the binary format deserialization so uh security exception has been already referenced by Sdi in one of his white papers regarding the potential things that exist in .NET Framework that lead to Binary formatter deserialization again it's very hard to use it again during the serialization because if you have to chain some different kind of a serializer and deserializer so I have explained all of this in details in the white paper so I'm not don't want to focus on that right now you only have to be aware that there is such a gadget and also I will later show you how you can abuse it in a different way also it's here I've got an example of a gadget in Apache nms active mq Library where again the get body getter leads to the binary format deserialization so I was able to abuse it in one of the products and in here we've got an example of this serialization to serialization gadget so it exists in Amazon Amazon AWS SDK core Library which is pretty popular and in here during the deserialization we are reading the content of the file but when we are not able to benefit from it if there is only this deserialization right but if this object is getting again serialized we are receiving such a json right and as you can see the json contains the content of the file right so if we are able to change the serialization and deserialization we can have us such a gadget you know for doing a file read so right now I would like to quickly show you an example for infras device master so as I have mentioned the settings property value Gadget could be used against message pack so in here I'm running the exploit we have reached the deserialization and nothing happened right but right now we are at this serialization phase and we are reaching the binary formatter so I was able to get the preal remote code execution by you know by just using the serialization gadget right I was not able to get a pwn for the deserialization but as the object was again serialized I was able to get the code execution directly for the serialization and it turned out to be nice right because I was not aware of you know any serialization gadgets so in fact you can use those serialization gadgets when there is an extensive block list for instance that you are not able to bypass or you are not aware of any Gadget that you can use against the target so another example of insecure

serialization is in solar winds rabbit and qu which I have analyzed in the beginning of the talk so I didn't mention it to you that you are able to force the solar winds information service to um serialize the object again after the this serialization if you if your object is stored in one of some solar winds based classes it serializes it again and it puts it to solar wind rabbit mq and you can extract that from the solar winds rabbit mq right so I was not able to find any direct remote code execution Ser realization gadgets here but we have those info! gadgets so in fact I was able to use three different gadgets here the first one was the gadget in the Amazon optimistic loog text file which I have already showed you solar winds uses this D so you could use that to do the file read another example is the internal solar winds Gadget which allowed me to leak database credentials so that's the stuff about you know the internal codebase and serialization gadgets a lot of products Implement like a class which has a getter which retrieves database credentials right so if you have insecure serialization you can use that to you know Le the entire for instance database connection string and do some stuff if it's available and the last Gadget was in fact the internal solar WIS serialization Gadget which also allowed you to perform the file read so you can see the code we have a Constructor and through the Constructor we are able to control the file name and there is a get contents method which just reads the content of the file and returns it right so my idea here U and and I I'm familiar with solar WIS I was aware of you know the file read assistant to rce primitive because solar winds um solar winds has an aank instance deployed on all interfaces and it's protected with some secret that is stored in the file but if you have the secret you can just connect to the erlank and execute anything you want right so it's the last demo for the serialization so I'm running the X exploit in the solar winds and you can see that right now we are at the dis realization phase so during the dis realization we are reaching the Constructor for this SQL file script right and we are trying to retrieve the file which stores the airline cookie and we have reached the serialization so during the serialization the file content is being extracted and if we continue the exploit after some time we receive the file because the serialized payload was sent back to the solar wi rabbit mq right so I have just extracted the serialized payload so here's the cookie so I'm I right now I can just you know take this cookie to connect to the solar winds Aang and execute any comment so that's an example of another you know like non-direct rce Gadget but if you have nothing else to use so you are just desperate you are looking for anything and you never know what you find right so that's it for the insecure serialization but you know what if I would like to use insecure serialization Gadget but there is no serialization so I was thinking about that and you may be aware that Alvaro Munos and Alexander miros has already presented a long time ago one arbitrary getter call Gadget right so in fact our insecure Sur realization gadgets are based on the getter call and my idea is maybe we can you know just change the arbitrary gter call Gadget with our insecure serialization Gadget to trigger like the serialization like Behavior directly through the deserialization so it turns out that the presented Gadget does not work anymore at least for me I was not able to use that you can find the details in the white paper so I decided to look for new arbitrary getter call gadgets and I found four of them in NET Framework property grid combo box list box and checkl box Gadget so the entire idea for it is like you have an insecure serialization Gadget you take the arbitrary getter call Gadget you chain them together and you are deserializing the serialization

gadget and then you call the malicious getter right during the dist realization of course it has some limitations in contrary to the you know regular serialization so you cannot use them for infol file read and stuff because you are not getting the object back right the serialized object back um I assume that vendors are going to start blocking it in the future so you may need to look for new gadgets or wait for just something that performs the serialization directly and for json.net it does that work for type name handling set for a member only so you have to be aware of that uh if you are interested in technical details why it does not work you can check them in the white paper so here is an example of Gadget so we are using the property grid arbitrary getter call Gadget and chain it with this security exceptions to realization Gadget so we can see that in the first part of the payload we are defining the property G Gadget and to its selected objects member which is a list of objects we provide the security exception serialization Gadget in the method member we Define a binary formatter Gadget and we pass that to Jon net so json.net basically you know deserializes this security exception through its serializable Constructor and it sets this method related member to our binary formatter Gadget then it's going to pass it to the property grid selected objects Setter which in fact leads to the getter call right it will call our get method which leads to Binary formatter and we can use that to get the remote code execution another example is the combo box chain with settings property value Gadget so in here you can see that we have a combo box Define defined it has an items member and in here we are defining the settings property value serialization Gadget and in the serialized value member we are provided our binary format Gadget and then we just say to combo box call the get property value so it will you know deserialize settings property value and then it will call its get property value getter and again we are getting in the code execution by chaining arbitrary get call Gadget and the insecure serialization Gadget so what I want you to know is that you can look for those you know insecure serialization Gadgets in the product that you're targeting and chain it with those arbitrary getter call Gadget and you are extending your entire exploitation surface right because you don't only have to look for those you know this realization Gadget you can look for Dangerous getter calls and just chain them right also very interesting Gadget it exists in the third party Library Microsoft Azure Cosmos it's a deserialization gadget which triggers serialization so during the der der serialization you're providing any object and this library is going to serialize it with json.net so if you have a lot a block list which blocks a lot of stuff you can have it in mind like you can trigger the serialization direct directly to the this through the dis serialization and you know perform another chain of gadgets and something like that so I believe it's an edge case but you never know what may be useful for you in the future right so in general I have found a couple new deserialization gadgets in NET Framework so we've got this chain of arbitrary getter call gadgets and insecure serialization gadgets also there are there is another rce gadget called exam image info it's a pretty comp one so I have described it in details in the white paper you can have a look because uh it accepts an extra type in this Constructor and you can abuse it with Jason doet for instance but uh it's not so easy and it's quite an interesting interesting technical case also I have found you know non rce gadgets like picture box which performs server side request forgery so you can use it sometimes like URL DNS for Java but but it's a little bit more powerful because it actually allows you to perform the request and you can use

different protocols you can use HTTP you can use use FTP you can use file so anything right so I'm going to skip some image info you can find Details in the white paper and I just want to quickly mention that if you are aware of a lot of different gadgets and you can change them you can bypass block lists pretty quickly because in here you've got a patch for this infras device master after I have exploited it with this insecure serialization so as you can see the settings property value which I have used against it was blocked right but I didn't care because I've just taken the arbitrary getter call Gadget with property grid and I have mentioned that there is insecure serialization Gadget in one of Apache libraries and this library was used in the products so I just chained them I've chained you know this getter call Gadget and insecure serialization Gadget to bypass this block this in like two minutes right so the more gadgets you know the more you know the biggest is your attack surface you know how to chain them the easier it is to you know P those targets and the last thing that I would like to show you are the gadgets in Net 5 six and seven because as Community we are not aware of many gadgets for those versions of net I mean they are not frequently used in products right now so that's why we are not looking for them but in general if WPF is enabled for Net 5 and bigger because it's not by default you can just use object data provider it works right but I believe that it's been frequently blocked by everything because it's one of the most popular dot realization gadgets but I have found two remote dll loading gadgets based activation Factory and compiler results so the base activation Factory in fact leads to the kernel 32 load like Library xw so while looking for the serialization or serialization gadgets also take native C calls you know into the consideration because there are a lot of native calls in net and in the third party libraries so of course we can provide UNC puff here and load remote file or chain it with some f right functionalities but here we have very fun gadget called compiler results and it's in fact insecure serialization Gadget right because we have a getter and we have the assembly load file call here and we fully control the path so um you know it's it was pretty weird for me to find out that in net 4 as you may know the remote D loading was blacked for assembly load file load for methods and any different similar methods right so I was like this gadget is not very interesting right we can of course chain it uh with the arbitrary getter call Gadget the the same gadgets that I have shown you work also for Net 5 and higher but you need to have WPF enabled right so without WPF enabled you can use this as a serialization gadget with WPF enabled you can chain arbitrary get call Gadget and this but regarding the remote dll loading you know I was like it should be blocked right but I still try it and I was like what the actual hell it worked I was able to load remotely load dll through the assembly load file right and I was like what happened so I've checked the documentation for Net 7 and the documentation clearly says that you should not be able to remotely load dlls with those method so we have reported it to Microsoft and after some time Microsoft report told us that it's an expected behavior for those versions of net and they are just going to change the documentation right as you can imagine it never happened at least I have checked like two weeks ago and documentation for net 76 still says that you should not be able to load remote DLS with those method but you can so it's like you know deprecation of the security feature so if you're targeting those versions of doet have that in mind that you can load remote DLS with assembly methods okay so just to finish I would like to tell you that I have implemented all those stuff to serial.net so we

have like five new gadgets in the mainline of yo serial.net I have also implemented text formatting run properties for json.net and I have implemented three three plugins so the first plugin implements arbitrary getter call gadgets so you can provide any insecure serialization Gadget that you have found for instance in the product codebase and use this plugin to chain it with this selected arbitrary get call Gadget I have also implemented a plugin with gadgets for third party libraries so the remote DL loading Gadget and stuff and also I have implemented a plugin for net framework and non rce gadgets so for instance the server side request fory gadgets so if you are planning to look for some Gadgets in the future right and you have any Gadget in third party Library I encourage you to you know to commit to those plugins so the entire pull request is going to happen on Monday when I got get back from the conference also the white paper is going to published on Monday so sorry for that you have to wait for the to days when we get back from the conference so thank you very much for the attention I hope you like it and I hope that you will find This research useful during your your research thanks